

Etap 3: Implementacja i testowanie zapytań SQL (5 godz.)

Cel etapu

Przygotowanie kompletnych skryptów SQL, które realizują logikę biznesową systemu. Na tym etapie "ożywiamy" projekt, implementując zaawansowane zapytania, skrypty zasilające oraz weryfikując poprawność działania więzów i procedur w PostgreSQL.

Struktura skryptów

Zalecane jest zorganizowanie kodu w czytelne pliki SQL:

1. `schema.sql` : Definicja struktury (DDL), w tym tabele, klucze, więzy oraz procedury/funkcje/wyzwalacze z Etapu 2.
2. `seed.sql` : Skrypt zasilający bazę danymi testowymi.
3. `queries.sql` : Zestaw zapytań realizujących konkretne funkcjonalności aplikacji (np. wyszukiwanie, raporty).

Zadania na tym etapie

☐ Inicjalizacja bazy (PostgreSQL):

- ☐ Stworzenie skryptu `schema.sql`.
- ☐ Użycie `GENERATED ALWAYS AS IDENTITY` dla kluczy głównych.
- ☐ Przygotowanie raportu z atrybutami (zgodnie z wytycznymi - `information_schema.columns`).

☐ Zasilenie bazy danymi (Seeding):

- ☐ Skrypt `seed.sql` dodający: min. 20 filmów, 10 użytkowników i 50 wypożyczeń.
- ☐ Zadbanie o różnorodność danych (różne statusy, daty).
- ☐ Przygotowanie prezentacji zawartości tabel (`SELECT * FROM ...`).

☐ Implementacja logiki operacyjnej (w `queries.sql`):

☐ Moduł Użytkownika:

- ☐ Wyszukiwanie filmów (np. po tytule lub gatunku z użyciem `ILIKE`).
- ☐ Proces wypożyczenia (użycie transakcji `BEGIN; ... COMMIT;`).

☐ Moduł Administratora:

- ☐ Zarządzanie katalogiem (operacje CRUD na filmach).
- ☐ Panel statystyk (podstawowe zliczenia, np. najpopularniejsze filmy).

Obsługa błędów i więzów danych

W czystym SQL błędy są zgłaszane przez system bazy danych. Należy przetestować reakcję bazy na niepoprawne dane:

Kod błędu (SQLState)	Przyczyna	Oczekiwany komunikat / Wynik
23505 (Unique)	Próba rejestracji na zajęty e-mail.	Błąd naruszenia klucza unikalnego.
23503 (FK)	Próba usunięcia filmu, który jest wypożyczony.	Błąd więzów klucza obcego.
23514 (Check)	Cena filmu jest mniejsza niż 0.	Błąd naruszenia więzów CHECK .
P0001 (Raise)	Wyzwalacz z Etapu 2 zablokował akcję.	Własny komunikat z RAISE EXCEPTION .

Przykład implementacji zapytań (SQL)

```
-- 1. Wyszukiwanie filmów
SELECT tytuł, cena
FROM film
WHERE dostepny = TRUE
    AND tytuł ILIKE '%Matrix%';

-- 2. Proces wypożyczenia (Logika transakcyjna)
BEGIN;

INSERT INTO wypozyczenie (uzytkownik_id, film_id, data_wypozyczenia)
VALUES (1, 5, NOW());

-- Dodatkowa logika (np. zmiana statusu filmu)
UPDATE film SET dostepny = FALSE WHERE id = 5;

COMMIT;
```

Wymagania techniczne (Checklist)

- ☐ Czy przygotowałeś tabele z atrybutami dla każdej z 5 encji?
- ☐ Czy przygotowałeś tabele z zawartością dla każdej z 5 encji?
- ☐ Czy wszystkie zapytania INSERT/UPDATE zmieniające stan bazy są przemyślane pod kątem transakcji?
- ☐ Czy używasz zapytań parametryzowanych w kontekście potencjalnej aplikacji (np. ? lub :param)?

- ☐ Czy skrypty są czytelnie skomentowane?
- ☐ Czy po uruchomieniu skryptu `schema.sql` baza jest gotowa do pracy?
- ☐ Czy skrypt `seed.sql` można uruchomić wielokrotnie bez błędów (np. użycie `TRUNCATE` lub `ON CONFLICT DO NOTHING`)?